Retro Highway — Project Report

1. Project Overview

Retro Highway is a browser-based endless highway game implemented using HTML5 Canvas, CSS, and vanilla JavaScript.

The game uses an 8-lane road and challenges the player to avoid continuously generated traffic. The distinguishing feature is that traffic generation is not purely random. The implementation maintains short-term information about the player's behaviour and uses this information to influence later traffic patterns.

The project also contains accessibility checks intended to reduce the chance of generating an unreachable traffic configuration, together with anti-camping logic that increases pressure on repeatedly preferred areas of the road.

The implementation was developed with substantial assistance from large language models (LLMs). LLMs were used as coding assistance during implementation and iteration. The project therefore represents an AI-assisted software development workflow in which generated code was integrated, modified, tested, and organized into a working game.

---

2. Objectives

The project was designed around several objectives:

1. Build a playable retro-style endless highway game in the browser.
2. Implement traffic patterns rather than relying entirely on independent random vehicle placement.
3. Track basic player behaviour.
4. Adapt later traffic patterns using observed behaviour.
5. Prevent generated patterns from becoming impossible to navigate.
6. Increase difficulty as the player's score grows.
7. Provide persistent high scores and basic audio/settings controls.
8. Deliver the project as a self-contained client-side application.

---

3. System Architecture

The game can be viewed as the following control loop:


Keyboard Input
Player State
Behaviour Tracking
- lane usage
- movement history
- camping tendency
Adaptive Pattern Generation
Accessibility / Movement Constraints
Traffic Flow / Movement
Collision Detection
Game Over

```
              |
              v
+--------------------------+
| Score & Difficulty       |
+------------+-------------+
              |
              v
           Next Frame
```

The application is implemented as a single-page client-side game. Rendering and state updates are performed on the browser's animation loop.

---

4. Game State

The main game state includes:

- current animation frame
- lane offset
- pause state
- game-start state
- game-over state
- player state
- traffic/enemy state
- score
- high score
- pattern buffer
- player behaviour statistics
- audio state
- volume state

The high score is stored using browser `localStorage`.

---

5. Road and Lane Model

The game defines:

- a fixed canvas of 530 × 600 pixels;
- road boundaries inside the canvas;
- **8 lanes** evenly distributed across the playable road.

The player's movement is lane-based. Pressing the left or right arrow changes the player's target lane, and the player's sprite interpolates toward the target horizontal position.

This produces responsive movement while keeping the game mechanics tied to the lane structure used by the traffic generator.

---

6. Player Behaviour Tracking

The adaptive system maintains a `playerBehaviorData` structure.

It tracks:

Preferred lane usage

The implementation increments a counter for the player's current lane.

This creates a simple empirical distribution of where the player tends to drive.

Movement history

The last 20 observed lane positions are maintained.

This gives the system short-term information about recent movements.

Recent position history

The last 10 positions are used to estimate how much the player moves between lanes.

Camping tendency

Variance in the recent lane positions is used as a heuristic measure of camping.

A low variance means the player has remained in a relatively narrow part of the road and therefore receives a higher camping-tendency value.

Adaptive score

The system also maintains an internal adaptive score. The increment becomes faster after the game score exceeds 300.

These values are behavioural heuristics. They are not outputs of a trained machine-learning model.

---

7. Adaptive Traffic Pattern Generation

The main generator is:

```
generateAdaptivePattern()
```

The function considers:

- current player lane;
- preferred side;
- most avoided lanes;
- current score;
- early/late game state;
- recent gap density;
- allowable movement distance.

It generates a binary lane pattern in which a lane is either occupied or open.

The number of generated vehicles is randomized within a controlled range rather than being fixed.

---

8. Pattern Generation Strategy

`generateVehiclePositions()` uses several stages.

Stage 1 — Target preferred side

At higher difficulty, the generator may place vehicles on the side of the road the player tends to prefer.

Stage 2 — Block avoided lanes

The generator can also target lanes that the player rarely uses.

An important constraint is that it attempts to leave at least one escape option.

Stage 3 — Clustering

Vehicles can be grouped into clusters to create denser traffic arrangements.

Stage 4 — Fill remaining positions

Unfilled positions are selected from the remaining available lanes.

The result is more structured than independent random lane selection.

---

9. Accessibility Constraints

The game contains several functions intended to ensure that generated patterns remain navigable:

```
 enforceMovementConstraints()
ensureGapContinuity()
forcePatternAccessibility()
validatePatternAccessibility()
```

Movement constraint

`enforceMovementConstraints()` checks whether a gap exists within the number of lanes the player can reasonably reach.

If no such gap exists, the function clears a lane to restore a reachable path.

Gap continuity

`ensureGapContinuity()` supports continuity between successive traffic patterns.

Accessibility validation

`validatePatternAccessibility()` checks the generated pattern against the configured movement constraints.

These mechanisms are rule-based safeguards, not a formal proof that every possible game state is solvable.

---

10. Anti-Camping Behaviour

If the player displays a strong camping tendency after the early game, `applyAntiCampingMeasures()` uses the player's preferred lanes to make the preferred region more challenging.

This gives the game a feedback loop:

```
Player behaviour
      ↓
```

```
Lane preference
      ↓
Traffic bias
      ↓
Player changes behaviour
      ↓
New traffic bias
```

The purpose is to reduce the effectiveness of staying in a single comfortable lane for long periods.

---

11. Traffic Speed and Movement

The game includes several layers of traffic-speed logic.

Global difficulty

`getSpeedMultiplier()` maps score progression onto a bounded speed range.

The implementation clamps the score to a configured maximum and interpolates from a minimum to a maximum speed multiplier.

Per-vehicle variation

`chooseVehicleSpeed()` assigns each vehicle a randomized speed factor.

The factor varies by ±25% around the base speed within configured limits.

Collision-aware speed adjustment

`adjustSpeedForCollision()` reduces a proposed speed when a vehicle is approaching another vehicle too closely.

IDM-style update

`updateIDM()` computes an acceleration-like update for traffic movement.

The system then updates each vehicle's speed and vertical position.

This provides more variation in traffic movement than moving every vehicle with an identical constant velocity.

---

12. Scoring and Game Over

A normal scoring vehicle contributes to the player's score when it leaves the bottom of the canvas.

Non-scoring roadside obstacles such as:

- barriers
- boxes
- cones

are excluded from score increments.

A collision between the player and a traffic object triggers `triggerGameOver()`.

The final score is then shown on the game-over screen.

The best score is stored using `localStorage`.

---

13. Rendering

Rendering is performed on an HTML5 Canvas.

The `draw()` function invokes:

1. sidewalk rendering;
2. road rendering;
3. roadside object rendering;
4. enemy rendering;
5. player rendering.

The road uses dashed lane dividers, while sprite images are used for the player, vehicles, roadside objects, and obstacles.

---

14. Audio and User Interface

The game loads three background audio tracks and selects among them.

The interface provides:

- start screen;
- high-score display;
- settings menu;

- pause/continue;
- home navigation;
- music toggle;
- volume control;
- restart;
- game-over display.

The music state and related preferences are maintained through browser storage where applicable.

---

15. File Structure

```
Retro-Highway/
├── index.html
├── style.css
├── game.js
├── README.md
├── REPORT.md
│
├── assets_audio/
│   ├── Audioinsmusic - Basscape.mp3
│   ├── Audioinsmusic - Driftline.mp3
│   └── Audioinsmusic - Groovoid.mp3
│
└── assets_images/
    ├── highway.png
    ├── player_images/
    ├── sidewalk_images/
    └── traffic_images/
```

Main files

`index.html` - defines the user interface and canvas; - loads the stylesheet and JavaScript; - defines audio and settings controls.

`style.css` - controls the retro visual design; - styles the menus, buttons, score display, game-over screen, and canvas.

`game.js` - contains the complete game state and gameplay logic.

The previous duplicate JavaScript file was removed from the cleaned repository so that there is only one authoritative implementation.

---

16. Technology Choices

HTML5 Canvas

Canvas was used because the game primarily consists of continuously rendered 2D sprites and road geometry. It avoids the need for a larger rendering framework.

Vanilla JavaScript

The game is implemented without a framework, which keeps the runtime lightweight and makes the game state and animation loop explicit.

LocalStorage

Local storage provides simple client-side persistence for data such as the high score without requiring a backend.

CSS

The visual layer uses CSS to create a pixel-inspired interface around the canvas.

---

17. AI-Assisted Development

LLMs were used substantially during implementation.

The development model was:

```
 Game requirement / feature idea
             ↓
LLM-assisted implementation
             ↓
Integration into game
             ↓
Testing and debugging
             ↓
Behaviour refinement
```

```
              ↓
Final source organization
```

The LLM assistance covered code generation and iteration, but the final application depends on the integration of multiple systems:

- player state
- lane model
- traffic generation
- behaviour tracking
- accessibility constraints
- traffic movement
- collision handling
- rendering
- audio/settings management

The project should therefore be described as an **LLM-assisted software project**.

The adaptive traffic system itself is **not machine learning**. It uses explicit heuristics and hand-designed rules based on player behaviour.

This distinction is important: using an LLM to assist with programming does not make the game an ML system.

---

18. Testing Considerations

The project was tested as an interactive browser game during development.

Important behaviours to verify when modifying the implementation include:

Navigation

- player can move between all eight lanes;
- player cannot move outside road boundaries.

Traffic

- traffic continues to spawn;
- multiple vehicle types render;
- traffic movement continues when the game is active;
- non-scoring obstacles do not increment the score.

Adaptation

- preferred lane statistics update;
- camping tendency changes with player movement;
- adaptive patterns are generated after the early game;
- anti-camping rules activate when the threshold is exceeded.

Accessibility

- generated patterns retain at least one reachable gap where required;
- transition between traffic patterns does not trivially remove all escape routes.

State management

- pause works;
- restart works;
- home navigation works;
- high score persists;
- music/volume controls work.

---

19. Limitations

The current implementation has several limitations.

Heuristic adaptation

The player model is a small rule-based heuristic system. It does not learn a predictive model of the player.

Solvability guarantee

The accessibility functions are safeguards, not a formal mathematical guarantee that every possible sequence of generated patterns is solvable.

Asset loading

Some image objects are constructed inside rendering functions, which may be improved by preloading and caching image objects once rather than recreating them during drawing.

Desktop-oriented input

The current control system is based on keyboard arrow keys and is not designed as a mobile/touch interface.

Single-page architecture

All major game logic is contained in `game.js`. A larger game could benefit from separating modules for:

- game state
- player
- traffic
- pattern generation
- rendering
- audio
- UI

---

20. Possible Future Improvements

Potential improvements include:

1. Separate the large `game.js` file into ES modules.
2. Preload and cache all image assets.
3. Add automated unit tests for pattern generation and accessibility rules.
4. Add deterministic seeded pattern generation for reproducible debugging.
5. Measure traffic-generation performance and animation-frame timing.
6. Improve the formal definition of pattern solvability.
7. Add mobile/touch controls.
8. Add gameplay telemetry for quantitative evaluation of the adaptive system.
9. Add a lightweight development/debug overlay showing generated patterns and accessibility checks.
10. Replace heuristic thresholds with parameterized configuration so difficulty can be tuned without editing core logic.

---

21. Conclusion

Retro Highway is a self-contained browser game that goes beyond basic random obstacle spawning.

Its main technical feature is the feedback loop between the player's observed lane behaviour and later traffic generation. The system combines:

```
 Player modelling
+
Pattern generation
+
Constraint checking
+
Adaptive difficulty
+
Traffic dynamics
+
Real-time rendering
```

The project also demonstrates an AI-assisted software-development workflow in which LLMs were used to accelerate implementation and iteration while the final application was assembled and refined as a coherent interactive system.

The most important technical distinction is that **LLM assistance was used for development, while the game's adaptive behaviour is implemented with explicit programmatic heuristics rather than machine learning**.